

# 40 — Definition of Done

Version 1.0 | Engineering Quality & Release Contract | September 2026

*A feature is done only when it is implemented, secure, testable, observable, recoverable, governed, and evidenced.*

# Document Control & Quality Contract

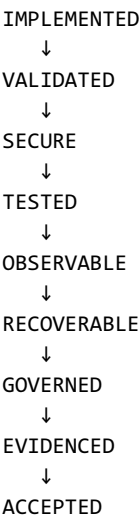
| Field                  | Value                                                                                                                       |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Document               | CloudShield AI Definition of Done                                                                                           |
| Identifier             | 40                                                                                                                          |
| Version                | 1.0                                                                                                                         |
| Status                 | Engineering baseline                                                                                                        |
| Scope                  | Entire CloudShield AI product, including DAIL as a bounded subsystem                                                        |
| Specification baseline | 00–39                                                                                                                       |
| Primary cloud          | AWS for MVP                                                                                                                 |
| Purpose                | Define objective completion criteria for features, components, vertical slices, releases, infrastructure, AI, DAIL, and MVP |
| Core rule              | No implementation item is considered complete merely because its code works on the happy path                               |

## Executive Summary

This document is the final quality contract for the CloudShield AI implementation program. It defines what 'done' means at multiple levels: task, feature, component, API, data model, AI capability, DAIL capability, vertical slice, infrastructure change, release, recovery exercise, and MVP. The contract prevents the project from declaring completion based solely on code compilation or a successful demonstration.

CloudShield AI follows a trust-oriented completion model: functionality must be correct; security and tenant isolation must be enforced; state transitions must obey lifecycle invariants; AI output must remain non-authoritative; DAIL verification must be exact-version-bound; governed cloud changes must be authorized and reconciled; evidence and audit must be durable; observability must exist; performance/reliability requirements must be measured; and deployment/recovery paths must be proven.

## Canonical Definition of Done



No stage may be skipped for a completion claim.

## 1. Purpose and Scope

- Define mandatory completion criteria for all CloudShield AI engineering work.
- Create a consistent quality gate across backend, frontend, infrastructure, AI, DAIL, integrations, security, testing, and operations.
- Prevent 'works on my machine', happy-path-only, demo-only, or unobservable implementations from being treated as complete.
- Provide objective evidence requirements for phase gates G0–G10 from Specification 39.
- Establish the conditions for MVP acceptance and production readiness.
- Preserve the distinction between CloudShield AI as the product and DAIL as a bounded verification subsystem.

## 2. Definition of Done Levels

| Level                    | Definition                                    | Required outcome                                                             |
|--------------------------|-----------------------------------------------|------------------------------------------------------------------------------|
| L0 Task                  | Small implementation unit                     | Code/config/documentation complete and locally validated                     |
| L1 Feature               | User-visible or domain capability             | Functional + secure + tested + observable + documented                       |
| L2 Component             | Reusable service/module/subsystem             | Contracts + failure behavior + operational characteristics proven            |
| L3 Vertical Slice        | End-to-end workflow                           | Integrated behavior proven across all required boundaries                    |
| L4 Infrastructure Change | Terraform/AWS/deployment change               | Security, drift, rollback, observability, and recovery validated             |
| L5 AI Capability         | LLM-powered capability                        | Grounded, structured, guarded, evaluated, non-authoritative                  |
| L6 DAIL Capability       | DAIL verification/decision-support capability | Exact-version binding, invariants, evidence, failure handling                |
| L7 Release Candidate     | Potential production version                  | Full regression, security, performance, DR, deployment and rollback evidence |
| L8 MVP Release           | Product milestone                             | Core control loop works safely and all critical acceptance gates pass        |

## 3. Universal Definition of Done

Every implementation item must satisfy the following baseline unless an approved exception explicitly removes a criterion.

| Dimension      | Done means                                                               |
|----------------|--------------------------------------------------------------------------|
| Requirement    | Traceable to one or more approved specifications/requirements.           |
| Design         | Architecture/domain implications reviewed; no hidden boundary violation. |
| Implementation | Code/configuration is complete and maintainable.                         |
| Correctness    | Expected behavior and relevant negative paths work.                      |
| Security       | Authorization, tenant isolation, secrets, input validation,              |

|                  |                                                                                                     |
|------------------|-----------------------------------------------------------------------------------------------------|
|                  | and threat considerations are addressed.                                                            |
| State            | Lifecycle transitions and invariants are enforced server-side.                                      |
| Testing          | Appropriate automated and integration tests pass.                                                   |
| Failure handling | Timeout, retry, idempotency, degradation, and reconciliation behavior are defined where applicable. |
| Observability    | Logs/metrics/traces/events exist at appropriate boundaries.                                         |
| Audit/Evidence   | Required actions and control decisions produce durable records.                                     |
| Performance      | Relevant latency/throughput/resource impact is measured or bounded.                                 |
| Documentation    | API/schema/runbook/configuration documentation is updated.                                          |
| Deployment       | CI/CD gates pass and artifact/configuration is reproducible.                                        |
| Recovery         | Backup/rollback/recovery implications are known and tested where relevant.                          |
| Acceptance       | Explicit acceptance criteria are passed with evidence.                                              |

## 4. Requirement & Traceability DoD

- Every feature/task has a stable identifier.
- Each requirement maps to a specification section or approved product decision.
- Acceptance criteria are written before implementation is declared complete.
- Security and reliability requirements are included, not appended after functional development.
- Dependencies on other workstreams are explicit.
- Out-of-scope behavior is documented where ambiguity could create unsafe assumptions.
- Changes to requirements are recorded through the project's change-control/ADR process.
- Implementation evidence links the requirement to tests, code/configuration, and release artifacts.

## 5. Design & Architecture DoD

- Component boundaries match the system architecture and bounded contexts.
- CloudShield AI remains the product boundary; DAIL remains a bounded subsystem.
- External providers are accessed through defined adapters/interfaces.
- LLM vendor SDKs are isolated behind the AI abstraction layer.
- Cloud provider SDKs are isolated behind cloud integration adapters.
- Long-running operations use durable asynchronous jobs where required.
- Critical state transitions are deterministic and protected by lifecycle rules.
- Authorization is enforced at the domain/application boundary, not only in the UI.
- Data ownership and source-of-truth semantics are explicit.
- Failure and degradation behavior is part of the design before implementation is accepted.

## 6. Domain, Identity & Lifecycle DoD

| Area           | Mandatory completion criteria                                                                        |
|----------------|------------------------------------------------------------------------------------------------------|
| Identity       | Stable provider-backed resource identity; display names are not identity.                            |
| Tenant         | All tenant-scoped data and commands enforce tenant boundaries.                                       |
| Versioning     | Mutable control objects use version/concurrency semantics.                                           |
| Lifecycle      | Allowed state transitions are explicit and server-enforced.                                          |
| Invalidation   | Material changes invalidate dependent results where specified.                                       |
| Unknown state  | Partial/stale/unknown data remains explicit.                                                         |
| Dependencies   | Safety-relevant edges have provenance/confidence.                                                    |
| Impact         | Impact analysis binds to exact candidate version and relevant graph state.                           |
| Trusted state  | Can only advance after required verification, governance, promotion, and authoritative confirmation. |
| Reconciliation | Ambiguous external outcomes have explicit reconciliation state.                                      |

## 7. Security Definition of Done

- Authentication boundary is implemented and tested.
- Authorization is enforced for every protected command/query.
- Tenant isolation has automated positive and negative tests.
- Least-privilege identities are used for AWS access and promotion.
- Secrets are not stored in source code, logs, artifacts, or unprotected configuration.
- Sensitive data is redacted from operational telemetry.
- Input validation and output validation are implemented.
- Web/API security controls are active.
- Dependency, secret, SAST, container/image, and IaC security scanning pass at the required gate.
- Threat-model implications are addressed.
- Security-relevant actions produce audit records.
- AI prompt injection, data exfiltration, tool misuse, and unsafe output scenarios are tested.
- DAIL promotion authority is isolated from ordinary application and AI workloads.
- Critical security findings block completion unless formally risk-accepted.

## 8. Testing Definition of Done

| Test layer  | Completion requirement                                 |
|-------------|--------------------------------------------------------|
| Unit        | Business rules and deterministic logic covered.        |
| Component   | Module behavior and failure paths covered.             |
| Contract    | API/provider/external boundaries conform to contracts. |
| Integration | Database, queue, AWS, storage, AI and relevant         |

|             |                                                                                       |
|-------------|---------------------------------------------------------------------------------------|
|             | integrations tested.                                                                  |
| System      | Cross-component workflows pass.                                                       |
| E2E         | Relevant user/business journey passes end-to-end.                                     |
| Security    | Threat controls and isolation tested.                                                 |
| AI          | Correctness, grounding, structure, safety, regression and failure behavior evaluated. |
| DAIL        | Invariants, exact-version binding, evidence and negative paths tested.                |
| Performance | Relevant workload meets or is explicitly bounded against Specification 38.            |
| Resilience  | Failure injection/recovery tested for critical paths.                                 |
| DR          | Applicable RTO/RPO and restore/reconciliation paths tested.                           |
| Regression  | Existing protected behavior remains passing.                                          |

## 9. API Definition of Done

- Request/response schemas are defined and versioned.
- Authentication and authorization are enforced.
- Tenant filtering is server-side.
- Validation errors and domain errors use the standard error contract.
- Pagination/filtering/sorting behavior is bounded and documented.
- Idempotency exists for commands where retries can cause duplicate side effects.
- Optimistic concurrency is enforced for mutable critical resources.
- Lifecycle transitions cannot be set arbitrarily by clients.
- Long-running work returns durable job references rather than blocking indefinitely.
- Rate limits and dependency timeouts are defined.
- API metrics, traces, audit events, and relevant evidence are emitted.
- OpenAPI/API documentation and integration tests are updated.
- Backward compatibility impact is assessed before release.

## 10. Data & Persistence Definition of Done

- Schema changes have migrations.
- Migration forward path is tested.
- Rollback/recovery implications are documented.
- Indexes support required access patterns.
- Transactions cover required atomic state changes.
- Network calls are not performed inside long-lived DB transactions.
- Retention/classification requirements are implemented.
- Large evidence/artifacts use object storage according to architecture.
- Data authorization is enforced at query/command boundaries.
- Backups include the data required for the defined recovery tier.
- Data integrity constraints and reconciliation checks exist for critical state.

## 11. AWS / Cloud Integration Definition of Done

- AWS access uses the defined secure connection and role-assumption model.
- Credentials/secrets are handled through approved secret mechanisms.
- Discovery handles provider pagination, throttling, retries, and timeouts.
- Provider resource identity is deterministic.
- Observations contain timestamps/provenance/freshness metadata.
- Partial discovery is represented explicitly.
- Cloud API errors map to defined domain outcomes.
- Provider mutations are isolated behind the cloud integration interface.
- Mutation preconditions are checked before execution.
- Ambiguous mutation outcomes enter reconciliation.
- Post-mutation authoritative observation is performed where required.
- AWS integration tests cover supported resource types and failure modes.
- Cloud permissions are least privilege and environment-specific.

## 12. AI / LLM Definition of Done

- Capability is invoked through the provider-neutral AI abstraction layer.
- Provider credentials are isolated from domain code.
- Input/context is authorized and minimized.
- External/untrusted content is treated as potentially adversarial.
- Prompt-injection defenses are active.
- Output uses a defined structured schema where machine-consumed.
- Output is validated before candidate/domain creation.
- Grounding/evidence references are preserved where required.
- Uncertainty is represented rather than hidden.
- Model failure/timeouts/malformed outputs have safe degraded behavior.
- AI cannot directly verify, approve, promote, or create trusted state.
- AI task has evaluation coverage under Specification 37.
- Latency, reliability, token/cost behavior are measurable.
- Model/provider configuration is versioned and traceable.
- Local/open-weight model support is optional unless explicitly included in the release.

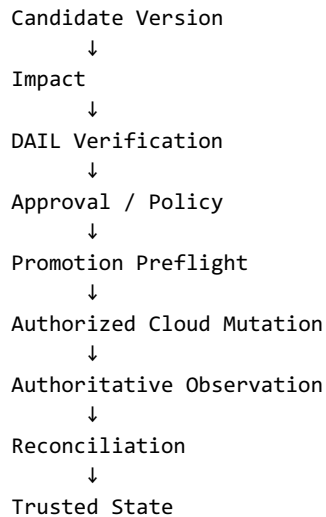
## 13. DAIL Definition of Done

DAIL completion criteria apply to DAIL itself as a subsystem. They do not redefine CloudShield AI as a DAIL-only product.

- DAIL consumes canonical CloudShield domain contracts rather than duplicating conflicting identity semantics.
- Verification is bound to the exact candidate version.
- Verification inputs have known freshness/provenance.
- Required invariants are deterministic and testable.
- Verification cannot be bypassed through API/UI shortcuts.
- Partial verification cannot satisfy a full required gate.
- Verification produces durable results and evidence.
- Verification failure/block/retry/reconciliation states are explicit.
- DAIL promotion authority is isolated from ordinary application/AI workloads.
- DAIL performance is measured without weakening verification depth.
- DAIL experiments cannot directly create production trusted state.

- DAIL changes are deployed through the same secure CI/CD and observability model.
- DAIL evidence is linked to candidate, inputs, run, software/configuration versions, and outcome.

## 14. Remediation, Governance & Promotion DoD



- Candidate content is immutable for the verification/approval unit.
- Impact analysis is complete for required scope.
- DAIL verification is successful and bound to the candidate version.
- Required approval/policy decision exists.
- Promotion preflight rechecks critical preconditions.
- Promotion uses least-privilege identity.
- External mutation is idempotent where provider semantics allow.
- Ambiguous outcomes are reconciled before retry.
- Before/after state and execution evidence are durable.
- Trusted state is advanced only after all required gates.
- Rejected, blocked, failed, and reconciled outcomes remain auditable.

## 15. Evidence & Audit Definition of Done

| Evidence type         | Done requirement                                                                                             |
|-----------------------|--------------------------------------------------------------------------------------------------------------|
| Audit event           | Actor, action, target, time, outcome, correlation context, and relevant version identifiers.                 |
| Cloud evidence        | Authoritative provider response/observation or durable reference to it.                                      |
| Verification evidence | Checks executed, inputs/versions, results, and failure details.                                              |
| AI evidence           | Model/config version, request class, grounding/context references, validation outcome where policy requires. |
| Promotion evidence    | Candidate, approval, execution, before/after state, reconciliation.                                          |
| Deployment evidence   | Commit, artifact digest, deployment ID, environment, outcome.                                                |
| Benchmark evidence    | Dataset/workload, software/configuration, environment,                                                       |

|                   |                                                                                    |
|-------------------|------------------------------------------------------------------------------------|
|                   | metrics, safety outcomes.                                                          |
| Recovery evidence | Incident/recovery action, restored components, reconciliation, validation results. |

Operational logs, metrics, and traces are not substitutes for required audit or evidence records.

## 16. Observability Definition of Done

- Logs are structured and correlation-aware.
- Metrics exist for critical request, job, dependency, provider, and state-transition paths.
- Distributed traces cover important cross-component workflows.
- Async jobs expose queue wait, execution, retry, and terminal state.
- Critical lifecycle events are observable.
- Security events and audit events are observable separately from general logs.
- Dashboards exist for service health and critical control-loop health.
- Alerts cover critical SLO/SLA burn, saturation, reconciliation backlog, and silent-failure risk.
- Telemetry redaction is tested.
- Observability failure does not silently suppress critical audit/evidence.

## 17. Performance & Reliability Definition of Done

| Area             | Required evidence                                                                           |
|------------------|---------------------------------------------------------------------------------------------|
| API              | Relevant p50/p95/p99 measurements; normal read p95 baseline $\leq 500$ ms where applicable. |
| Async acceptance | Durable acceptance p95 baseline $\leq 1$ s for defined mutating async path.                 |
| Jobs             | Durability $\geq 99.9\%$ for defined valid workload.                                        |
| Availability     | MVP baseline $\geq 99.5\%$ monthly excluding planned maintenance.                           |
| Promotion        | No silent failure; terminal or reconciliation outcome.                                      |
| Audit            | Critical event durable recording $\geq 99\%$ baseline.                                      |
| Load             | Representative normal/peak workloads tested.                                                |
| Stress           | Saturation threshold and graceful degradation understood.                                   |
| Soak             | No unacceptable resource leakage.                                                           |
| Fault injection  | Critical failure paths validated.                                                           |
| Tenant isolation | No material noisy-neighbor violation.                                                       |
| Recovery         | RTO/RPO targets from Specification 33 validated for applicable tiers.                       |

## 18. Product & UX Definition of Done

- User workflow is complete for the intended feature.
- Loading, empty, stale, partial, unavailable, error, and degraded states are represented.

- Destructive or cloud-affecting actions require appropriate confirmation/governance.
- Trusted state is visually distinct from proposed/unverified state.
- AI recommendations are distinguishable from authoritative findings/state.
- Verification status and evidence are discoverable.
- Permissions and unauthorized states are handled safely.
- Pagination and bounded data loading are used for large datasets.
- Accessibility requirements are met for the applicable release scope.
- UX tests cover the golden path and important failure states.

## 19. Infrastructure & DevOps Definition of Done

- Terraform is the canonical infrastructure definition.
- Terraform plan/apply is controlled and auditable.
- Environment isolation is implemented.
- CI/CD uses approved federated identities where supported.
- Artifacts are immutable/versioned.
- Security scanning gates are active.
- Database migrations are part of release strategy.
- Deployment health checks and smoke tests exist.
- Rollback strategy is documented and tested.
- Deployment outcomes are observable and auditable.
- DR/rebuild implications are known.
- DAIL promotion authority is isolated in deployment/runtime permissions.
- Configuration and secrets are environment-safe and version-controlled where appropriate.

## 20. External Integration Definition of Done

- Integration capability and permission scope are explicitly defined.
- Tenant boundary is enforced.
- Authentication/credential handling is secure.
- Data mapping and versioning are documented.
- Idempotency/deduplication is implemented where necessary.
- Timeout, retry, rate-limit, and dead-letter behavior exists.
- External outages do not corrupt core CloudShield state.
- Webhooks verify authenticity where supported.
- External content entering AI workflows passes trust/injection controls.
- Integration actions cannot bypass CloudShield lifecycle, DAIL, governance, evidence, or trusted-state rules.

## 21. Documentation Definition of Done

- Requirement/acceptance criteria are traceable.
- API contracts are documented.
- Data/schema changes are documented.
- Configuration variables and secrets references are documented without exposing secrets.
- Operational runbooks are updated.
- Failure/recovery behavior is documented.
- Architecture decisions are recorded where behavior or boundaries changed.
- User-facing help text is updated where applicable.

- Release notes identify material behavior changes.
- Evidence links or references are retained for gate review.

## 22. Code Quality Definition of Done

- Code follows repository conventions.
- Static analysis/lint/format checks pass.
- No known critical/high-severity code-quality defect remains without approval.
- Dead code and debug paths are removed.
- Error handling is explicit.
- External calls have timeouts.
- Retry loops are bounded.
- Concurrency behavior is intentional.
- Secrets and sensitive data are not logged.
- Tests are deterministic and do not rely on uncontrolled external state.
- Complex safety-critical logic has explanatory comments/design documentation where needed.

## 23. Definition-of-Done Exceptions

Exceptions are allowed only when explicitly governed; they must not be used to hide unfinished safety-critical work.

| Exception type               | Requirement                                                                                                |
|------------------------------|------------------------------------------------------------------------------------------------------------|
| Temporary technical debt     | Owner, reason, impact, due date, and tracking item required.                                               |
| Test environment limitation  | Exact untested scope and production risk documented.                                                       |
| External provider limitation | Provider behavior, mitigation, and degraded mode documented.                                               |
| MVP scope deferral           | Capability explicitly classified as post-MVP; no dependency hidden.                                        |
| Known performance gap        | Measured gap, safe operating envelope, and remediation plan documented.                                    |
| Security exception           | Formal risk acceptance by authorized security/governance owner; critical unsafe bypasses are not waivable. |
| DR gap                       | Recovery limitation documented with business impact and remediation plan.                                  |

No exception may permit unauthorized promotion, tenant-isolation failure, evidence fabrication, DAIL bypass, or creation of trusted state without required authoritative confirmation.

## 24. Gate-Level Definition of Done

| Gate              | Completion contract                                                                   |
|-------------------|---------------------------------------------------------------------------------------|
| G0 Build-Ready    | Repository, architecture, requirements, tooling, CI and implementation backlog ready. |
| G1 Platform-Ready | Tenant, authorization, persistence, jobs, audit, observability operational.           |

|                       |                                                                                 |
|-----------------------|---------------------------------------------------------------------------------|
| G2 Discovery-Ready    | AWS connection, discovery, identity, observations and reconciliation proven.    |
| G3 Analysis-Ready     | Security/risk analysis deterministic, evidenced and regression-tested.          |
| G4 Impact-Ready       | Dependency graph and candidate-bound impact analysis proven.                    |
| G5 AI-Ready           | AI abstraction, grounding, structured output, safety and evaluation proven.     |
| G6 Verification-Ready | DAIL exact-version verification, evidence and negative paths proven.            |
| G7 Control-Loop-Ready | Governance, promotion, cloud mutation, reconciliation and trusted state proven. |
| G8 Product-Ready      | UX, jobs, evidence, reporting and selected integrations complete.               |
| G9 Production-Ready   | Security, performance, resilience, DR, rollback, operations proven.             |
| G10 MVP-Accepted      | Complete MVP evidence package reviewed and accepted.                            |

## 25. Vertical-Slice Definition of Done

| Slice                        | Done criteria                                                                           |
|------------------------------|-----------------------------------------------------------------------------------------|
| VS-1 Tenant/API/DB/Audit     | Authorized tenant-scoped operation persists correctly and produces audit.               |
| VS-2 AWS Discovery           | Controlled AWS account produces stable assets and observations.                         |
| VS-3 Finding/Risk            | Configuration generates deterministic finding, risk, and evidence.                      |
| VS-4 Dependency/Impact       | Candidate-bound impact is calculated with explicit graph provenance.                    |
| VS-5 AI Candidate            | AI produces validated recommendation; candidate creation is safe and traceable.         |
| VS-6 DAIL Verification       | Exact candidate version passes/fails deterministic verification with evidence.          |
| VS-7 Promotion/Trusted State | Governed authorized mutation executes, reconciles, and advances trusted state safely.   |
| VS-8 Drift                   | Authoritative drift invalidates affected trusted state and triggers required follow-up. |

## 26. Release Candidate Definition of Done

1. All release requirements are mapped to acceptance criteria.
2. All automated quality/security gates pass.
3. Critical and high-priority regression suites pass.
4. Supported AWS integration tests pass.
5. AI evaluation regression suite passes.

6. DAIL verification suite passes.
7. Tenant-isolation and authorization tests pass.
8. Performance baseline is within approved envelope.
9. Fault-injection scenarios pass for critical workflows.
10. Backup/restore and reconciliation tests pass.
11. Deployment and rollback tests pass.
12. Observability dashboards and alerts are active.
13. Runbooks are current.
14. Evidence/audit completeness is verified.
15. Release artifact is immutable and traceable to source.

## 27. Production Readiness Definition of Done

- Gate G9 has passed.
- Gate G10 review package is substantially complete.
- Production AWS environment is hardened.
- Least-privilege production identities are verified.
- Secrets/KMS and rotation procedures are operational.
- Backups and recovery paths are tested.
- SLO dashboards and alerts are operational.
- Security operations and incident response runbooks exist.
- On-call/ownership responsibilities are defined.
- Known failure modes have runbooks.
- Critical reconciliation backlog is zero before launch or explicitly accepted.
- Cloud mutation scope is explicitly limited and audited.
- Rollback procedure has been rehearsed.
- Post-deployment smoke and reconciliation checks are defined.

## 28. MVP Definition of Done

MVP is complete only when the product demonstrates a safe, evidence-backed control loop for the explicitly supported AWS scope.

- CloudShield AI can securely onboard a supported AWS environment.
- It discovers and reconciles supported assets.
- It records authoritative observations with freshness/provenance.
- It evaluates supported security rules and produces findings/risk.
- It maintains supported dependency relationships and computes candidate impact.
- It uses AI for explanation/recommendation through the abstraction layer without granting authority.
- It creates immutable remediation candidates/versions.
- DAIL verifies the exact candidate version.
- Governance/approval controls production-affecting changes.
- A least-privilege promotion path executes supported mutations.
- Ambiguous outcomes are reconciled.
- Authoritative cloud state is re-observed.
- Trusted state advances only after required conditions.
- Drift invalidates affected trusted state.
- Evidence/audit are durable and queryable.
- Performance/reliability baselines are met or formally accepted.

- Security operations and DR are operational.
- Deployment/rollback is reproducible.
- No critical unresolved safety/security/correctness defect exists without explicit authorized acceptance.

## 29. Required Evidence Package

| Evidence artifact        | Required contents                                             |
|--------------------------|---------------------------------------------------------------|
| Requirements matrix      | Specification → requirement → implementation → test → status  |
| Architecture record      | Architecture baseline + ADRs + material deviations            |
| Test report              | Unit/component/integration/system/E2E/security results        |
| AI evaluation report     | Quality, grounding, safety, latency, cost, regression         |
| DAIL verification report | Verification runs, invariants, evidence, outcomes             |
| Performance report       | Workloads, p50/p95/p99, throughput, saturation, resource use  |
| Resilience report        | Fault injection, recovery, reconciliation outcomes            |
| DR report                | RTO/RPO, restore, data integrity, reconciliation              |
| Security report          | Scans, threat-model closure, isolation tests, exceptions      |
| Deployment report        | Commit, artifact digest, deployment, smoke, rollback evidence |
| Operational readiness    | Dashboards, alerts, runbooks, ownership                       |
| Release approval         | Gate status, open exceptions, risk acceptance, final decision |

## 30. Completion Anti-Patterns

| Anti-pattern           | Why it is not Done                                                      |
|------------------------|-------------------------------------------------------------------------|
| Code compiles          | Compilation proves almost nothing about behavior or safety.             |
| Happy path works       | Failure, security, and recovery behavior remain unknown.                |
| UI is finished         | Backend authorization/lifecycle controls may still be incomplete.       |
| AI gives a good answer | AI quality does not grant authority or prove grounding.                 |
| DAIL returns PASS      | PASS is invalid if bound to wrong/stale candidate inputs.               |
| Terraform applies      | Infrastructure can still be insecure, unobservable, or unrecoverable.   |
| Tests are green        | Tests may omit critical negative paths and production-scale conditions. |
| Logs exist             | Logs are not equivalent to durable audit/evidence.                      |
| Cloud change succeeded | Without authoritative reconciliation, final state may be unknown.       |
| Benchmark is fast      | Speed is not acceptable if it weakens safety or correctness.            |
| No errors seen         | Absence of observed failure is not evidence of resilience.              |

|                       |                                                                      |
|-----------------------|----------------------------------------------------------------------|
| Feature is documented | Documentation cannot substitute for implementation and verification. |
|-----------------------|----------------------------------------------------------------------|

## 31. Master Done Checklist

- ☐ Requirement traced
- ☐ Architecture reviewed
- ☐ Domain/lifecycle impact reviewed
- ☐ Authorization implemented
- ☐ Tenant isolation verified
- ☐ Secrets handled safely
- ☐ Implementation complete
- ☐ Unit tests passing
- ☐ Component/contract tests passing
- ☐ Integration tests passing
- ☐ System/E2E tests passing
- ☐ Negative/failure paths tested
- ☐ Observability implemented
- ☐ Audit/evidence implemented
- ☐ Performance impact measured
- ☐ Security scans passing
- ☐ Deployment path passing
- ☐ Rollback/recovery path addressed
- ☐ Documentation updated
- ☐ Acceptance evidence attached
- ☐ Open exceptions explicitly governed
- ☐ Feature accepted by responsible owner

## 32. Master Definition of Done Acceptance Criteria

AC-01: Every feature and release has explicit, testable completion criteria.

AC-02: Done criteria cover functionality, security, testing, observability, reliability, evidence, and deployment.

AC-03: Tenant isolation and authorization are mandatory completion gates.

AC-04: Lifecycle and state invariants are enforced server-side.

AC-05: AI remains non-authoritative and passes defined evaluation/safety gates.

AC-06: DAIL verification is exact-version-bound and cannot be bypassed.

AC-07: Governed cloud promotion requires authorization, verification, evidence, and reconciliation.

AC-08: Trusted state cannot be created from stale, partial, unknown, or ambiguous evidence.

AC-09: Critical external outcomes reach terminal or reconciliation state.

AC-10: Performance and reliability requirements are measured under representative workloads.

AC-11: DR, rollback, and recovery paths are tested for applicable critical components.

AC-12: Security/quality CI gates are part of completion, not optional pre-release cleanup.

AC-13: Exceptions are explicit, owned, time-bounded where appropriate, and governed.

AC-14: MVP acceptance requires an evidence package covering the entire control loop.

AC-15: Gate G10 is the final authority for MVP completion.

### 33. Final Engineering Principle

**CloudShield AI is not done when it can perform the action. It is done when the system can perform the action correctly, safely, observably, recoverably, accountably, and repeatedly under the conditions it claims to support.**

**The product earns trust through evidence—not through successful demos.**

## **CloudShield AI — Definition of Done v1.0**

*Implemented • Validated • Secure • Tested • Observable • Recoverable • Governed • Evidenced • Accepted*